Universidad de Granada

ETSIIT Informática y Telecomunicación

Dpto. de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Práctica 2

Técnicas de Búsqueda basadas en Poblaciones
para el Problema de Portfolio

Curso 2025–2026

Nombre: Jose Angel Carretero Montes
DNI: 25628892S
Email: joseangel1@correo.ugr.es
Grupo: Grupo 3
Horario: Martes 15:30–17:30

Índice

1. Descripción del Problema **2**

2. Aplicación de los Algoritmos al Problema **3**
2.1. Representación de soluciones . . . 3
2.2. Generación aleatoria de soluciones . . . 3
2.3. Operador de reparación (`fix`) . . . 3
2.4. Operador de selección: torneo (k=3) . . . 3
2.5. Operadores de cruce . . . 3
2.6. Operador de mutación . . . 4
2.7. Búsqueda Local corregida (usada en los AMs) . . . 5

3. Algoritmos Genéticos y Meméticos **6**
3.1. Algoritmo Genético Generacional (AGG) . . . 6
3.2. Algoritmo Genético Estacionario (AGE) . . . 7
3.3. Algoritmos Meméticos (AM) . . . 8

4. Estructura del Código y Manual de Usuario **10**

5. Experimentos y Resultados **11**
5.1. Casos del problema . . . 11
5.2. Resultados . . . 11
5.3. Boxplots . . . 12

6. Análisis de Resultados **13**
6.1. BLX vs Aritmético . . . 13
6.2. AGG vs AGE: influencia del tamaño del problema . . . 13
6.3. Algoritmos Meméticos: resultados y análisis . . . 14
6.4. Análisis de la Búsqueda Local . . . 14
6.5. Análisis de la variabilidad (σ) en AGs y AMs . . . 14
6.6. Análisis de los boxplots . . . 16
6.7. Capacidad de generalización (test 2025) . . . 17
6.8. Conclusiones generales . . . 17

7. Trabajo Voluntario: Estudio del parámetro α del operador BLX **18**
7.1. Resultados por dataset . . . 18
7.2. Boxplots del estudio . . . 19
7.3. Análisis . . . 20

1. Descripción del Problema

El **Problema de Portfolio** consiste en determinar la asignación óptima de capital entre N activos financieros con el objetivo de maximizar el retorno esperado minimizando simultáneamente el riesgo.

Representación de los datos

Para cada activo $i \in \{1, \ldots, N\}$ y día $t \in \{1, \ldots, T\}$ se dispone del retorno diario μ_{ti}, calculado como:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Los datos cubren el periodo 2015–2024 para entrenamiento y el año 2025 para evaluación.

Función objetivo

Dado un vector de pesos $w = (w_1, \ldots, w_N)$:

$$\text{fitness}(w) = \text{Beneficio}(w) - \lambda \cdot \text{Riesgo}(w)$$

$$\text{Beneficio}(w) = \sum_{i=1}^{N} w_i \cdot \sum_{t=1}^{T} \log(1 + \mu_{ti}) \qquad \text{Riesgo}(w) = \sqrt{\sum_{i=1}^{N} \sum_{j=1}^{N} w_i \cdot \Sigma_{ij} \cdot w_j}$$

siendo Σ_{ij} la covarianza entre activos i y j, y $\lambda = 500$.

Algorithm 1: Cálculo de fitness(w)

Entrada: pesos w, beneficio por activo *benefit_factor*[], matriz de covarianzas *cov*[][], λ
Salida: valor de fitness

```
benefit ← 0;
para i ← 1 to N hacer
    benefit ← benefit + w[i] · benefit_factor[i];
risk_sq ← 0;
para i ← 1 to N hacer
    para j ← 1 to N hacer
        risk_sq ← risk_sq + w[i] · cov[i][j] · w[j];
devolver benefit − λ · √risk_sq
```

Restricciones

$$\sum_{i=1}^{N} w_i = 1 \qquad w_i = 0 \ \vee \ lo \leq w_i \leq hi \quad \forall i$$

2. Aplicación de los Algoritmos al Problema

2.1. Representación de soluciones

Una solución es un vector $w \in \mathbb{R}^N$ donde w_i indica la fracción de capital invertida en el activo i: exactamente cero o en $[lo, hi]$.

2.2. Generación aleatoria de soluciones

Igual que en la Práctica 1: subconjunto aleatorio de activos activos, pesos aleatorios en $[lo, hi]$, normalización y reparación.

2.3. Operador de reparación (`fix`)

Tres fases: normalización, recorte de rango y redistribución iterativa. Es esencial tras los operadores de cruce, que pueden generar soluciones infactibles.

2.4. Operador de selección: torneo (k=3)

Se eligen $k = 3$ individuos al azar y se devuelve el de mayor fitness.

Algorithm 2: Torneo ($k = 3$)

Entrada: población P
Salida: individuo ganador

```
1 mejor ← P[aleatorio en [0, |P| − 1]];
2 para i ← 1 to 3 hacer
3 |   rival ← P[aleatorio en [0, |P| − 1]];
4 |   si rival.fitness > mejor.fitness entonces
5 |   |   mejor ← rival;
6 devolver mejor
```

2.5. Operadores de cruce

Cruce Aritmético

Para cada posición i se genera $r_i \in [0, 1]$ independiente:

$$\text{Off}_i^1 = r_i \cdot S_i^1 + (1 - r_i) \cdot S_i^2 \qquad \text{Off}_i^2 = r_i \cdot S_i^2 + (1 - r_i) \cdot S_i^1$$

Algorithm 3: Cruce Aritmético(S^1, S^2, problema)

Entrada: padres S^1, S^2; $[lo, hi]$
Salida: hijos Off^1, Off^2

$\text{Off}^1 \leftarrow S^1$; $\text{Off}^2 \leftarrow S^2$;
para $i \leftarrow 1$ **to** N **hacer**
  $r \leftarrow$ aleatorio en $[0, 1]$;
  $\text{Off}^1[i] \leftarrow r \cdot S^1[i] + (1 - r) \cdot S^2[i]$;
  $\text{Off}^2[i] \leftarrow r \cdot S^2[i] + (1 - r) \cdot S^1[i]$;
`fix(`Off^1`)`; `fix(`Off^2`)`; **devolver** Off^1, Off^2

Cruce BLX-α

Con $I_i = |S_i^1 - S_i^2|$:

$$hmin_i = \text{mín}(S_i^1, S_i^2) - \alpha \cdot I_i \qquad hmax_i = \text{máx}(S_i^1, S_i^2) + \alpha \cdot I_i$$

Ambos hijos se generan uniformemente en $[\text{máx}(0, hmin_i), \text{mín}(hi, hmax_i)]$, $\alpha = 0{,}3$.

Algorithm 4: Cruce BLX-α(S^1, S^2, problema), $\alpha = 0{,}3$

Entrada: padres S^1, S^2; $[lo, hi]$; α
Salida: hijos Off^1, Off^2

para $i \leftarrow 1$ **to** N **hacer**
  $I \leftarrow |S^1[i] - S^2[i]|$;
  $hmin \leftarrow \text{máx}(0,\ \text{mín}(S^1[i], S^2[i]) - \alpha \cdot I)$;
  $hmax \leftarrow \text{mín}(hi,\ \text{máx}(S^1[i], S^2[i]) + \alpha \cdot I)$;
  $\text{Off}^1[i] \leftarrow$ aleatorio en $[hmin, hmax]$;
  $\text{Off}^2[i] \leftarrow$ aleatorio en $[hmin, hmax]$;
`fix(`Off^1`)`; `fix(`Off^2`)`; **devolver** Off^1, Off^2

2.6. Operador de mutación

El operador base es el **intercambio**: transfiere el 15 % del peso de un gen a otro, preservando $\sum w_i = 1$ y los límites $[lo, hi]$ por construcción, sin necesidad de `fix()`.

Algorithm 5: Mutación-intercambio(ind, $gene_idx$, problema)

Entrada: individuo ind, índice $gene_idx$, $[lo, hi]$

$gene_j \leftarrow$ aleatorio en $[0, N - 2]$;
si $gene_j \geq gene_idx$ **entonces**
  $gene_j \leftarrow gene_j + 1$;
$delta \leftarrow 0{,}15 \cdot ind[gene_idx]$;
$delta \leftarrow \text{mín}(delta,\ ind[gene_idx] - lo)$;
$delta \leftarrow \text{mín}(delta,\ hi - ind[gene_j])$;
si $delta > \varepsilon$ **entonces**
  $ind[gene_idx] \leftarrow ind[gene_idx] - delta$;
  $ind[gene_j] \leftarrow ind[gene_j] + delta$;
  evaluar fitness(ind); $evals \leftarrow evals + 1$;

La forma en que se **invoca** este operador depende del esquema de evolución, no de si el algoritmo es genético o memético:

Esquema estacionario (AGE): probabilístico por individuo

Con probabilidad $p_m = 0{,}1$ por cada hijo generado:

Algorithm 6: Aplicar mutación en AGE(h, problema)

si *rand()* $\leq p_m$ **entonces**
    $gene_idx \leftarrow$ aleatorio en $[0, N-1]$;
    Mutación-intercambio(h, $gene_idx$, problema);

Esquema generacional (AGG y todos los AMs): esperanza matemática

Se aplica exactamente $n_{mut} = \lfloor p_m \cdot N_p \rfloor = 10$ mutaciones fijas por generación sobre la nueva población P', tanto en AGG-Arit, AGG-BLX como en AM-All, AM-Rand y AM-Best (todos comparten el mismo bucle generacional):

Algorithm 7: Aplicar mutaciones en AGG/AMs (esperanza matemática)

$n_{mut} \leftarrow \lfloor p_m \cdot N_p \rfloor$;
$pop_actual \leftarrow |P'|$;
para $k \leftarrow 1$ **to** n_{mut} ***y*** $evals < maxEvals$ **hacer**
    $chrom_idx \leftarrow$ aleatorio en $[0, pop_actual - 1]$;
    $gene_idx \leftarrow$ aleatorio en $[0, N-1]$;
    Mutación-intercambio($P'[chrom_idx]$, $gene_idx$, problema);

2.7. Búsqueda Local corregida (usada en los AMs)

En la Práctica 1, la BL aplicaba `fix()` sobre cada solución candidata, lo cual era subóptimo dado que el propio operador de movimiento podría preservar las restricciones. En la Práctica 2, la BL utiliza un **operador de intercambio** que preserva $\sum w_i = 1$ y los límites $[lo, hi]$ por construcción, eliminando la necesidad de `fix()` en cada paso.

El operador de intercambio funciona del siguiente modo: se selecciona un par aleatorio de índices (origen, destino), se calcula el mayor desplazamiento δ compatible con los límites, y si $\delta > 0$ se transfiere esa cantidad de peso de origen a destino.

Algorithm 8: BL($sol_inicial$, problema, maxEvals, $p = 0{,}15$)

Entrada: solución inicial, problema, maxEvals, proporción p
Salida: solución, fitness, evals

$Pares \leftarrow \{(i,j) : i \neq j,\ i,j \in [0, N-1]\}$ // $N(N-1)$ *pares*;
$sol \leftarrow sol_inicial$;
$fitness \leftarrow$ evaluar(sol); $evals \leftarrow 1$;
$mejora \leftarrow$ verdadero;
mientras $mejora$ **y** $evals < maxEvals$ **hacer**
  $mejora \leftarrow$ falso;
  Barajar $Pares$ aleatoriamente;
  para *cada* $(origen, destino)$ *en* $Pares$ **y** $evals < maxEvals$ **hacer**
    $delta \leftarrow p \cdot sol[origen]$;
    $delta \leftarrow$ mín($delta,\ sol[origen] - lo$);
    $delta \leftarrow$ mín($delta,\ hi - sol[destino]$);
    si $delta \leq \varepsilon$ **entonces**
      continuar;
    $sol' \leftarrow sol$; $sol'[origen] \mathrel{-}= delta$; $sol'[destino] \mathrel{+}= delta$;
    $fit' \leftarrow$ evaluar(sol'); $evals \leftarrow evals + 1$;
    si $fit' > fitness$ **entonces**
      $sol \leftarrow sol'$; $fitness \leftarrow fit'$;
      $mejora \leftarrow$ verdadero; **romper**;
devolver sol, $fitness$, $evals$

3. Algoritmos Genéticos y Meméticos

3.1. Algoritmo Genético Generacional (AGG)

Parámetros

$N_p = 100$, $p_c = 0{,}8$, $p_m = 0{,}1$ (equivale a $n_c = 40$ cruces y $n_m = 10$ mutaciones fijas/generación), criterio de parada: 10 000 evaluaciones.

Esquema de evolución y reemplazamiento

El AGG usa la **esperanza matemática** tanto para el cruce como para la mutación: se calculan exactamente $n_{cruces} = \lfloor p_c \cdot N_p/2 \rfloor = 40$ pares que se cruzan, y los $N_p/2 - 40 = 10$ pares restantes producen copias directas. La mutación aplica exactamente $n_{mut} = \lfloor p_m \cdot N_p \rfloor = 10$ intercambios a individuos elegidos aleatoriamente.

Algorithm 9: AGG (generacional con elitismo, esperanza matemática)

Entrada: problema, maxEvals
Salida: mejor solución

Inicializar P con N_p aleatorios; $evals \leftarrow N_p$;
mientras $evals < maxEvals$ **hacer**
  $padres \leftarrow [\ \texttt{torneo}(P)\ \times N_p]$; $n_c \leftarrow \lfloor p_c \cdot N_p/2 \rfloor$; $P' \leftarrow \emptyset$;
  para $i \leftarrow 0$ **to** $N_p/2 - 1$ ***y*** $evals < maxEvals$ **hacer**
    $p_1 \leftarrow padres[2i]$; $p_2 \leftarrow padres[2i{+}1]$;
    si $i < n_c$ **entonces**
      $(h_1, h_2) \leftarrow \text{cruzar}(p_1, p_2)$; $evals \leftarrow evals + 2$;
    si no
      $h_1 \leftarrow p_1$; $h_2 \leftarrow p_2$;
    Añadir h_1, h_2 a P';
  $n_m \leftarrow \lfloor p_m \cdot N_p \rfloor$; **para** $k \leftarrow 0$ **to** $n_m - 1$ ***y*** $evals < maxEvals$ **hacer**
    $chr \leftarrow \text{rand}\ [0, |P'|{-}1]$; $g \leftarrow \text{rand}\ [0, N{-}1]$;
    Mutación-intercambio$(P'[chr], g)$;
  si máx$(P').fit <$ máx$(P).fit$ **entonces**
    mín$(P') \leftarrow$ máx(P);
  $P \leftarrow P'$;
devolver máx(P)

Se implementan dos variantes: **AGG-Arit** y **AGG-BLX**.

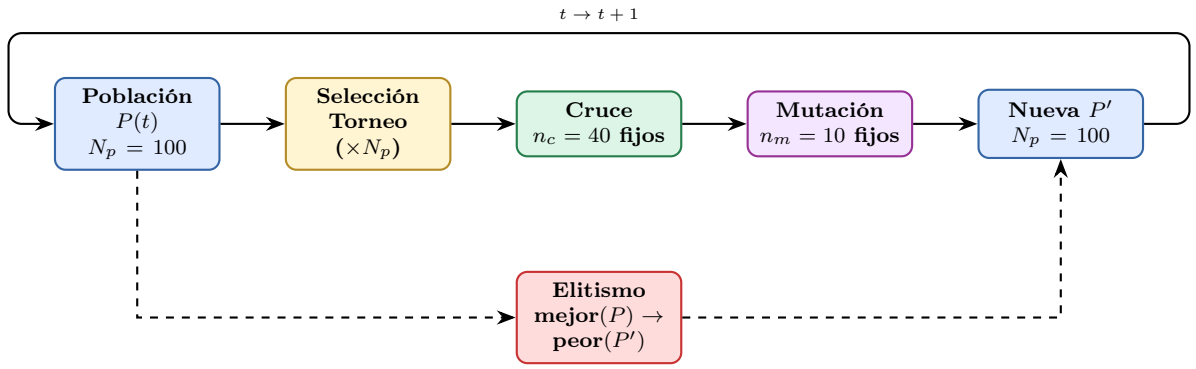


Figura 1: Ciclo evolutivo del AGG. Las líneas discontinuas muestran el flujo de elitismo.

3.2. Algoritmo Genético Estacionario (AGE)

Parámetros

$N_p = 100$, $p_c = 1{,}0$ (siempre se cruzan), $p_m = 0{,}1$ por individuo, criterio de parada: 10 000 evaluaciones.

Esquema de evolución y reemplazamiento

Algorithm 10: AGE (estacionario)

Entrada: problema, maxEvals
Salida: mejor solución

Inicializar P con N_p aleatorios; $evals \leftarrow N_p$;
mientras $evals < maxEvals$ **hacer**
  $p_1 \leftarrow$`torneo(`P`)`; $p_2 \leftarrow$`torneo(`P`)`;
  si $rand() \leq p_c$ **entonces**
    $(h_1, h_2) \leftarrow \text{cruzar}(p_1, p_2)$; $evals \leftarrow evals + 2$;
  si no
    $h_1 \leftarrow p_1$; $h_2 \leftarrow p_2$;
  si $rand() \leq p_m$ **entonces**
    $g \leftarrow \text{rand}[0, N-1]$; Mutación-intercambio$(h_1, g)$;
  si $rand() \leq p_m$ **entonces**
    $g \leftarrow \text{rand}[0, N-1]$; Mutación-intercambio$(h_2, g)$;
  $peor_1 \leftarrow$ mín(P);
  si $h_1.fit > peor_1.fit$ **entonces**
    $peor_1 \leftarrow h_1$;
  $peor_2 \leftarrow$ mín(P);
  si $h_2.fit > peor_2.fit$ **entonces**
    $peor_2 \leftarrow h_2$;
devolver máx(P)

El mín(P) *se recalcula para* h_2 *después de la posible inserción de* h_1.

Se implementan dos variantes: **AGE-Arit** y **AGE-BLX**.

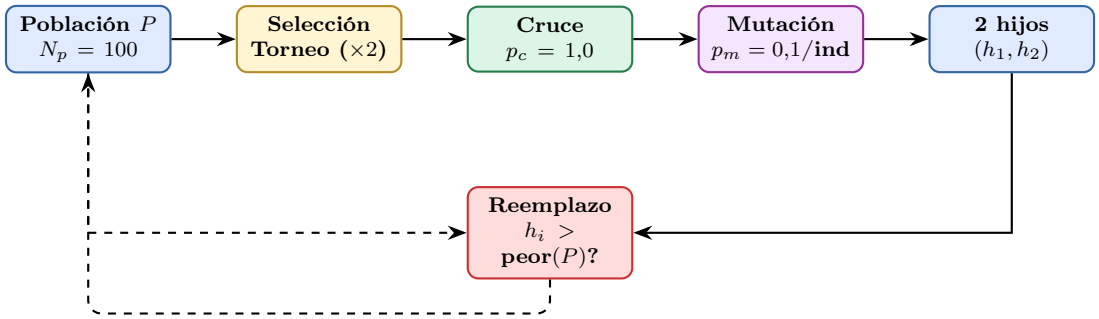


Figura 2: Ciclo evolutivo del AGE. Las flechas discontinuas muestran la consulta del peor individuo de P y la actualización en sitio: solo 0–2 miembros cambian por iteración.

3.3. Algoritmos Meméticos (AM)

Los AMs hibridan el AGG que mejor resultado ha proporcionado (**AGG-BLX**, véase Sección 5) con la BL de la Práctica 1 adaptada: fracción de movimiento $p = 0{,}15$ (en lugar de 0,40), máximo 100 evaluaciones por llamada. La BL se aplica **cada 10 generaciones**; el presupuesto de cada llamada es mín(100, maxEvals − *evals*) para respetar el límite global de 10 000 evaluaciones (incluidas las de la BL).

Los AMs comparten **exactamente el mismo bucle generacional que el AGG**: misma selección (N_p torneos, $k = 3$), mismo cruce con esperanza matemática ($n_c = 40$ pares fijos), mismo emparejamiento fijo, misma mutación con esperanza matemática ($n_m = 10$ fijos) y mismo elitismo. La única diferencia estructural es el bloque de BL que se activa cada 10 generaciones según el criterio de cada variante.

Parámetros comunes

$N_p = 100$, $p_c = 0{,}8$, $p_m = 0{,}1/N$ por gen (equivale a $n_m = 10$ mutaciones/generación).

AM-All: BL a todos los individuos

Algorithm 11: AM-All

```
si generaciones mód 10 = 0 entonces
    para i ← 1 to |P| y evals < maxEvals hacer
        budget ← mín(100, maxEvals − evals);
        P[i] ← BL(problem, P[i], budget); evals += bl.evals;
```

AM-Rand: BL aleatoria al 10 % de la población

Algorithm 12: AM-Rand

```
si generaciones mód 10 = 0 entonces
    para i ← 1 to |P| y evals < maxEvals hacer
        si rand() ≤ 0,1 entonces
            budget ← mín(100, maxEvals − evals);
            P[i] ← BL(problem, P[i], budget); evals += bl.evals;
```

AM-Best: BL a los $0{,}1 \cdot N_p$ mejores

Algorithm 13: AM-Best

```
si generaciones mód 10 = 0 entonces
    ordenar P descendente por fitness;
    para i ← 1 to mín(0,1 · |P|, |P|) y evals < maxEvals hacer
        budget ← mín(100, maxEvals − evals);
        P[i] ← BL(problem, P[i], budget); evals += bl.evals;
```

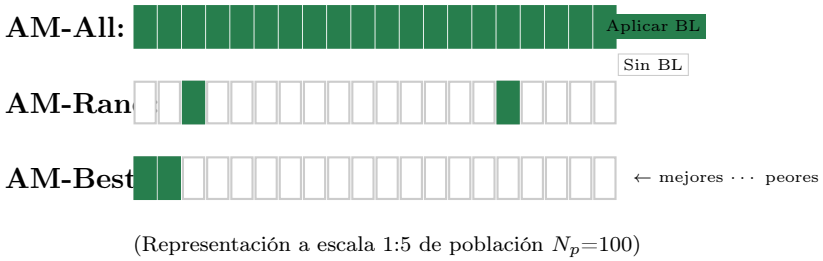


Figura 3: Comparación de estrategias de selección en AMs. Cada célula representa 5 individuos.

4. Estructura del Código y Manual de Usuario

```
portfolio/
  main.cpp               -- Experimentos principales (10 algoritmos x 3 datasets)
  main_blx_alpha.cpp     -- Trabajo voluntario: estudio del parámetro alpha en BLX
  CMakeLists.txt         -- Genera ejecutables: main y study_blx_alpha
  data/
    ibex_35.csv, syp_100.csv, syp_500.csv
  src/
    portfolioProblem.cpp -- Problema, fitness, fix, createSolution
    genetic.cpp          -- AGG, AGE, AM-All, AM-Rand, AM-Best
    localsearch.cpp      -- BL con operador intercambio (sin fix por vecino)
    greedy.cpp
    randomSearch.cpp
  inc/
    portfolioProblem.h, genetic.h, localsearch.h, greedy.h, randomSearch.h
  common/
    mh.h, problem.h, solution.h, random.hpp, util.h
```

Compilación y ejecución:

```
cmake -S . -B build
cmake --build build

./build/main [max_evals] [n_seeds]          # defecto: 10000 evals, 50 semillas
./build/study_blx_alpha [max_evals] [n_seeds]   # estudio alpha BLX

# Genera CSV: results_ibex_35.csv, results_syp_100.csv, results_syp_500.csv
#             blx_alpha_ibex_35.csv, blx_alpha_syp_100.csv, blx_alpha_syp_500.csv
python3 boxplot.py    # genera boxplots desde los CSV
```

5. Experimentos y Resultados

5.1. Casos del problema

Fichero	N	Días train	Días test	$[lo, hi]$
`ibex_35.csv`	30	2559	254	[0,005, 0,08]
`syp_100.csv`	97	2516	249	[0,005, 0,05]
`syp_500.csv`	457	2516	249	[0,005, 0,02]

Parámetros comunes: $\lambda = 500$, MAX_EVALS= 10 000, semillas 1–50.

5.2. Resultados

Cuadro 1: IBEX 35 — $N = 30$, $[lo, hi] = [0{,}005, 0{,}08]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
Greedy	−4,650	0,000	−3,387	0,000	0,115	1,0	0,000
Random	−4,941	0,070	−3,306	0,104	0,036	10000,0	0,021
BL	−4,573	0,019	−3,110	0,008	0,022	1257,0	0,001
AGG-Arit	−4,644	0,050	−3,132	0,039	0,043	10000,5	0,006
AGG-BLX	−4,525	0,018	−3,106	0,021	0,036	10000,4	0,008
AGE-Arit	−4,614	0,036	−3,124	0,038	0,041	10000,5	0,006
AGE-BLX	−4,520	0,012	−3,105	0,020	0,039	10000,6	0,009
AM-All	−4,725	0,054	−3,157	0,040	0,042	10000,0	0,020
AM-Rand	−4,491	0,001	−3,087	0,005	0,034	10000,2	0,011
AM-Best	−4,491	0,001	−3,084	0,003	0,033	10000,4	0,010

Cuadro 2: S&P 100 — $N = 97$, $[lo, hi] = [0{,}005, 0{,}05]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
Greedy	−3,664	0,000	−5,020	0,000	0,010	1,0	0,000
Random	−3,390	0,055	−4,113	0,193	0,080	10000,0	0,050
BL	−3,032	0,067	−3,870	0,033	0,097	5258,5	0,020
AGG-Arit	−3,112	0,040	−3,832	0,077	0,086	10000,4	0,021
AGG-BLX	−3,035	0,038	−3,774	0,045	0,087	10000,5	0,033
AGE-Arit	−3,158	0,054	−3,859	0,076	0,082	10000,6	0,022
AGE-BLX	−3,049	0,042	−3,795	0,063	0,089	10000,7	0,031
AM-All	−3,301	0,045	−3,990	0,065	0,080	10000,0	0,200
AM-Rand	−2,941	0,029	−3,735	0,030	0,091	10000,2	0,105
AM-Best	−2,912	0,025	−3,725	0,033	0,094	10000,5	0,100

Cuadro 3: S&P 500 — $N = 457$, $[lo, hi] = [0{,}005, 0{,}02]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
Greedy	−3,438	0,000	−5,205	0,000	−0,063	1,0	0,000
Random	−3,642	0,042	−4,430	0,167	0,012	10000,0	1,215
BL	−3,318	0,164	−4,228	0,119	0,008	9839,7	0,626
AGG-Arit	−2,961	0,073	−3,947	0,096	0,012	10000,5	0,367
AGG-BLX	−2,877	0,043	−3,920	0,083	0,014	10000,5	0,388
AGE-Arit	−3,201	0,069	−4,078	0,141	0,016	10000,6	0,360
AGE-BLX	−3,095	0,044	−3,990	0,084	0,020	10000,5	0,379
AM-All	−3,335	0,040	−4,174	0,092	0,011	10000,0	3,489
AM-Rand	−2,846	0,037	−3,901	0,075	0,018	10000,2	1,612
AM-Best	−2,811	0,031	−3,907	0,065	0,017	10000,4	1,485

5.3. Boxplots

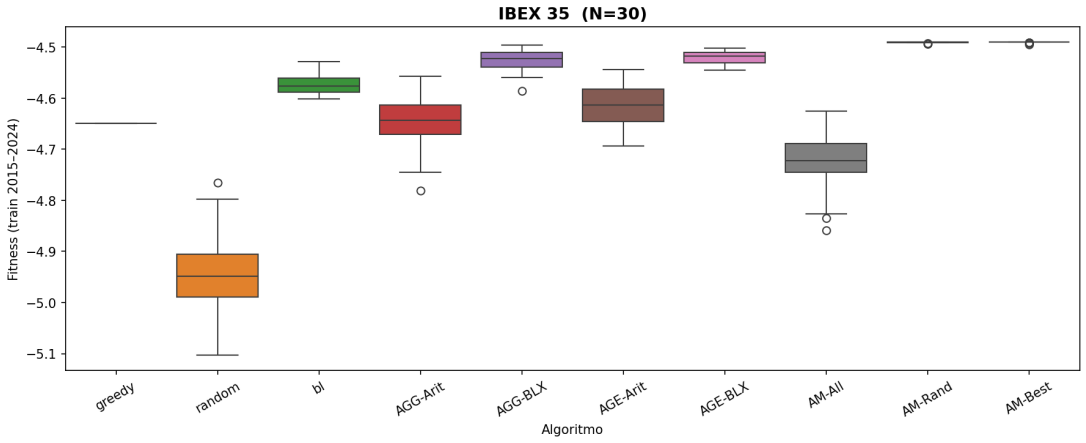


Figura 4: Distribución del fitness (2015–2024) en IBEX 35.

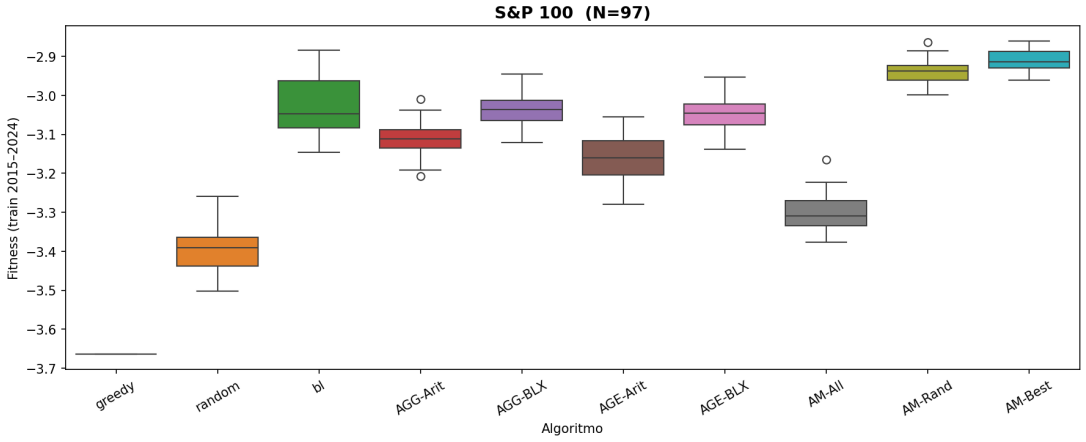


Figura 5: Distribución del fitness (2015–2024) en S&P 100.

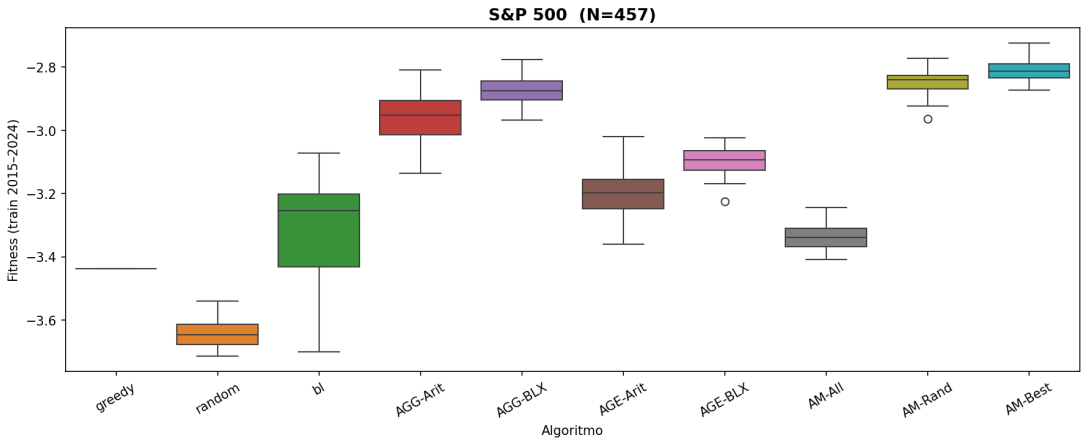


Figura 6: Distribución del fitness (2015–2024) en S&P 500.

6. Análisis de Resultados

6.1. BLX vs Aritmético

BLX-α supera al cruce Aritmético en las **6 comparaciones** (3 datasets $\times$ 2 esquemas AGG y AGE), sin excepción.

Dataset	AGG-Arit	AGG-BLX	AGE-Arit	AGE-BLX
ibex_35	−4,644	−4,525 (+0,119)	−4,614	−4,520 (+0,094)
syp_100	−3,112	−3,035 (+0,077)	−3,158	−3,049 (+0,109)
syp_500	−2,961	−2,877 (+0,084)	−3,201	−3,095 (+0,106)

El cruce Aritmético es puramente explotativo: genera hijos dentro del *convex hull* de los padres. BLX puede extenderse más allá de ese convex hull (controlado por α), permitiendo explorar nuevas regiones del espacio de soluciones. Esta capacidad de exploración resulta crítica tanto en espacios pequeños (escape de óptimos locales) como en espacios de alta dimensión (diversidad necesaria). Estos resultados justifican que los AMs usen **AGG-BLX** como base.

6.2. AGG vs AGE: influencia del tamaño del problema

Dataset	AGG-BLX	AGE-BLX	Ganador
ibex_35	−4,525	−4,520	AGE (+0,005)
syp_100	−3,035	−3,049	AGG (+0,014)
syp_500	−2,877	−3,095	AGG (+0,218)

El resultado más relevante es el de syp_500: **AGG supera a AGE por un margen de** 0,218, mientras que en los datasets pequeños las diferencias son mínimas.

La explicación reside en la dimensionalidad: en syp_500 ($N = 457$), cada generación del AGG genera 100 nuevos individuos que exploran el espacio de búsqueda de forma diversa, cubriendo una mayor región del espacio en cada iteración. El AGE, que solo produce 2 hijos por iteración, tiene un ritmo de exploración muy inferior en espacios de alta dimensión. Para ibex_35 ($N = 30$), el elitismo implícito del AGE (98 individuos

intactos por iteración) es más valioso que la exploración masiva del AGG con presupuesto limitado.

6.3. Algoritmos Meméticos: resultados y análisis

AM-Best y AM-Rand son los mejores algoritmos en todos los datasets, superando incluso a la Búsqueda Local:

Dataset	BL	AM-Rand	AM-Best
ibex_35	−4,573	−4,491 (+0,082)	−4,491 (+0,082)
syp_100	−3,032	−2,941 (+0,091)	−2,912 (+0,120)
syp_500	−3,318	−2,846 (+0,472)	−2,811 (+0,507)

La ventaja crece con N: en syp_500, AM-Best supera a BL en 0,507 unidades de fitness. El enfoque híbrido combina la exploración global de los AGs con la explotación local de la BL aplicada a los mejores individuos.

AM-All es el peor AM en todos los datasets. La causa es el agotamiento prematuro del presupuesto: AM-All aplica BL a los 100 individuos cada 10 generaciones, lo que supone $100 \times 100 = 10\,000$ evaluaciones en la primera aplicación (en la generación 10), agotando prácticamente todo el presupuesto. Apenas quedan evaluaciones para la evolución posterior. Los individuos, todavía inmaduros en la generación 10, quedan atrapados en óptimos locales por la BL prematura.

6.4. Análisis de la Búsqueda Local

Con los nuevos datos, la BL ya no es dominante. El cambio más notable es en syp_500: la BL alcanza −3,318 de media usando casi todo el presupuesto ($\approx$ 9840 evals), mientras que AGG-BLX logra −2,877 y AM-Best −2,811.

La razón es la **explosión del vecindario**: en syp_500 ($N = 457$), el número de pares posibles para el operador de intercambio es $457 \times 456 = 208\,392$. Con un límite de ≈ 9840 evaluaciones, la BL solo puede explorar menos del 5 % de una única pasada completa por el vecindario. En estas condiciones, la BL queda atrapada en el primer óptimo local que encuentra, sin capacidad de exploración global.

Los AGs y AMs, en cambio, mantienen una población diversa que explora múltiples regiones del espacio simultáneamente, siendo mucho más efectivos para N grande.

6.5. Análisis de la variabilidad (σ) en AGs y AMs

Los algoritmos de esta práctica (AGs y AMs) muestran patrones de variabilidad muy distintos entre sí, directamente relacionados con su diseño:

Variabilidad entre operadores de cruce

El cruce BLX produce consistentemente **menor varianza que el Aritmético** en ambos esquemas y los tres datasets:

	ibex_35		syp_100		syp_500	
	Arit σ	BLX σ	Arit σ	BLX σ	Arit σ	BLX σ
AGG	0,050	0,018	0,040	0,038	0,073	0,043
AGE	0,036	0,012	0,054	0,042	0,069	0,044

BLX, al explorar más allá del convex hull de los padres, guía la búsqueda hacia regiones más prometedoras de forma más consistente, reduciendo la sensibilidad a la semilla inicial. El cruce Aritmético, puramente explotativo, queda más atrapado en la cuenca de atracción de la solución inicial.

Variabilidad entre esquemas: AGG vs AGE

AGE presenta **menor σ que AGG** en ibex_35 y syp_100 para ambos operadores de cruce. El esquema estacionario, al reemplazar solo 0–2 individuos por iteración, mantiene una élite implícita grande que estabiliza la búsqueda. AGG, al regenerar toda la población cada generación, introduce mayor variabilidad generación a generación. En syp_500 la diferencia de σ es mínima entre AGG y AGE, dominando el efecto de la dimensionalidad.

Variabilidad de los AMs: efecto de la estrategia de BL

Es en los AMs donde la σ muestra las diferencias más extremas:

- **AM-Rand y AM-Best**: $\sigma \approx 0{,}001$ en ibex_35 — el resultado más llamativo de toda la práctica. Todas las semillas convergen al mismo óptimo independientemente del punto de inicio. La BL selectiva (aplicada a 10 individuos ya maduros evolutivamente) combina exploración global del AGG con explotación local precisa, encontrando siempre el mismo atractor. En syp_100 y syp_500 la varianza sube ($\sigma \approx 0{,}03$–$0{,}04$) pero sigue siendo la más baja de todos los algoritmos evolutivos.
- **AM-All**: paradójicamente, tiene la **mayor σ de los AMs** en syp_100 (0,045) y syp_500 (0,040), mayor incluso que algunos AGs puros. Al aplicar BL a los 100 individuos en generación 10 — cuando la población aún está inmadura —, el resultado queda fuertemente ligado a la calidad de esa población inicial, que varía mucho según la semilla. Los AGs puros son más estables porque no dependen de una fase de BL temprana.

Comparación de tiempos (AGs y AMs)

Algoritmo	**ibex_35 (s)**	**syp_100 (s)**	**syp_500 (s)**
AGG-Arit	0,006	0,021	0,367
AGG-BLX	0,008	0,033	0,388
AGE-Arit	0,006	0,022	0,360
AGE-BLX	0,009	0,031	0,379
AM-All	0,020	0,200	3,489
AM-Rand	0,011	0,105	1,612
AM-Best	0,010	0,100	1,485

Los AGs puros son los más rápidos: AGG y AGE tienen tiempos similares ya que ejecutan el mismo número de evaluaciones de fitness ($\approx$10 000) con la misma función de coste. BLX

es ligeramente más lento que Aritmético por el cálculo adicional de $[hmin_i, hmax_i]$ por posición.

AM-All es el AM más lento con diferencia (3,489s en syp_500 vs ≈0.38s de los AGs): aplica BL a los 100 individuos, y con syp_500 cada llamada a BL es costosa ($N = 457$, evaluación $O(N^2)$). AM-Rand y AM-Best son intermedios y similares entre sí, ya que ambos aplican BL a ≈10 individuos por fase memetica.

6.6. Análisis de los boxplots

Los boxplots (Figuras 4–6) permiten observar tanto la calidad media como la consistencia de cada algoritmo:

- **Greedy**: línea única sin caja (completamente determinístico). En ibex_35 su valor es competitivo (−4,650), pero en syp_100 y syp_500 aparece cortado en la parte inferior del gráfico, siendo el peor de todos: la heurística voraz colapsa en espacios de alta dimensión.
- **Random**: caja amplia con bigotes largos y outliers en los tres datasets, confirmando su alta sensibilidad a la semilla. En ibex_35 aparecen outliers superiores (semillas que por azar generan buenas combinaciones iniciales).
- **BL**: la **Búsqueda Local empleada en esta práctica es la versión corregida respecto a la Práctica 1**: el operador de intercambio preserva las restricciones por construcción, eliminando las llamadas innecesarias a `fix()` que distorsionaban el vecindario. Esta corrección es visible en la consistencia de los resultados.

  En **ibex_35** la caja es la más compacta del gráfico ($\sigma = 0{,}019$): BL converge siempre al mismo óptimo local en solo ≈1 257 evals, lo que confirma un paisaje de fitness suave con un atractor dominante. En **syp_100** la caja es moderada ($\sigma = 0{,}067$, converge en ≈5 258 evals) y la mediana es competitiva con los AGs. En **syp_500** ocurre lo contrario: **BL tiene la caja más ancha de todos los algoritmos** ($\sigma = 0{,}164$). La causa es estructural: con $N = 457$ el vecindario tiene $457 \times 456 = 208.392$ pares; con 10 000 evaluaciones BL solo puede explorar el 4,8 % de una única pasada, parando siempre por presupuesto (≈9 840 evals) y nunca por convergencia. El resultado depende de qué fracción aleatoria del vecindario exploró cada semilla, produciendo alta varianza. Los AGs tienen mucha menor varianza porque la población distribuye el presupuesto de forma intrínsecamente más robusta.
- **AGs**: varianza reducida y homogénea en todos los datasets. Las diferencias entre variantes son visibles: **BLX tiene medianas superiores a Aritmético** tanto en AGG como en AGE, confirmando la ventaja exploradora de BLX. En ibex_35 AGE-BLX supera ligeramente a AGG-BLX; en syp_500 la diferencia AGG-BLX − AGE-BLX es la más visible del gráfico (≈ 0,22 de fitness), mostrando que el esquema generacional escala mejor con N grande.
- **AM-All**: en ibex_35 su caja es la más grande de los AMs con un outlier hacia −4,89; en syp_100 es incluso **peor en mediana que todos los AGs puros** con varianza considerable. Esto refleja el agotamiento prematuro del presupuesto: aplicar BL a los 100 individuos en la primera fase memetica (generación 10) consume ≈10 000 evals de golpe, dejando sin presupuesto de evolución. La alta varianza indica que el resultado queda muy ligado a la calidad de la población en generación 10, que

depende mucho de la semilla.

- **AM-Rand y AM-Best**: en ibex_35 sus cajas son **prácticamente invisibles** ($\sigma \approx$ 0,001), con medianas claramente superiores a todos los demás algoritmos incluida la BL. En syp_100 y syp_500 mantienen varianza baja y medianas superiores. AM-Best tiene medianas ligeramente superiores a AM-Rand en todos los datasets, confirmando que dirigir la BL a los mejores individuos es más eficiente que aplicarla aleatoriamente.

6.7. Capacidad de generalización (test 2025)

Los rankings de entrenamiento se mantienen en test para los algoritmos más robustos. AM-Best generaliza bien en todos los datasets: ibex_35 test −3,084 (mejor de todos), syp_100 test −3,725 (mejor), syp_500 test −3,907 (mejor). AGG-BLX y AGE-BLX también generalizan correctamente, con test próximos a los de entrenamiento.

El caso más llamativo es **Greedy en syp_100 y syp_500**: el fitness de test (−5,020 y −5,205) es muy inferior al de entrenamiento (−3,664 y −3,438), indicando que la solución voraz sobreajusta al periodo histórico y no generaliza.

Algoritmo	Train (syp_500)	Test 2025 (syp_500)	Gap
Greedy	−3,438	−5,205	−1,767
BL	−3,318	−4,228	−0,910
AGG-BLX	−2,877	−3,920	−1,043
AM-Best	−2,811	−3,907	−1,096

6.8. Conclusiones generales

1. **BLX supera a Aritmético** en las 6 comparaciones (3 datasets × 2 esquemas): la exploración fuera del convex hull es crítica para la calidad.
2. **AGE supera a AGG** para N pequeño (ibex_35); **AGG supera a AGE** para N grande (syp_500, margen 0,218). La exploración masiva del AGG es decisiva en alta dimensión.
3. **AM-Best y AM-Rand son los mejores algoritmos globales**, superando a BL en todos los datasets. La hibridación evolutivo-local es la estrategia ganadora.
4. **AM-All fracasa** por agotamiento del presupuesto en la generación 10: $100 \times 100 = 10\,000$ evaluaciones de BL consumen todo el presupuesto sin evolución suficiente previa.
5. **BL no es competitiva para N grande**: la explosión del vecindario ($457 \times 456 = 208\,392$ pares en syp_500) impide explorar más del 5 % del vecindario con el presupuesto disponible.

7. Trabajo Voluntario: Estudio del parámetro α del operador BLX

Se ha realizado un estudio experimental del parámetro $\alpha \in \{0{,}0, 0{,}1, 0{,}2, 0{,}3, 0{,}5, 0{,}7, 1{,}0\}$ del operador de cruce BLX-α aplicado al algoritmo AGG-BLX, con 50 semillas por configuración. Se ha implementado un ejecutable dedicado `study_blx_alpha` (compilado junto con `main` mediante CMake) que ejecuta AGG-BLX con todos los valores de α sobre los tres datasets.

7.1. Resultados por dataset

Cuadro 4: Estudio de α en BLX — IBEX 35 ($N = 30$)

α	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,00	$-4{,}652 \pm 0{,}050$	$-3{,}132 \pm 0{,}033$	0,041
0,10	$-4{,}632 \pm 0{,}049$	$-3{,}124 \pm 0{,}037$	0,040
0,20	$-4{,}580 \pm 0{,}034$	$-3{,}113 \pm 0{,}028$	0,039
0,30	$-4{,}525 \pm 0{,}018$	$-3{,}106 \pm 0{,}021$	0,036
0,50	$-4{,}490 \pm 0{,}000$	$-3{,}083 \pm 0{,}002$	0,033
0,70	$-4{,}490 \pm 0{,}000$	$-3{,}083 \pm 0{,}002$	0,033
1,00	$-4{,}523 \pm 0{,}025$	$-3{,}113 \pm 0{,}027$	0,040

Cuadro 5: Estudio de α en BLX — S&P 100 ($N = 97$)

α	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,00	$-3{,}133 \pm 0{,}041$	$-3{,}857 \pm 0{,}080$	0,084
0,10	$-3{,}130 \pm 0{,}040$	$-3{,}849 \pm 0{,}068$	0,084
0,20	$-3{,}099 \pm 0{,}038$	$-3{,}826 \pm 0{,}051$	0,083
0,30	$-3{,}035 \pm 0{,}038$	$-3{,}774 \pm 0{,}045$	0,087
0,50	$-2{,}834 \pm 0{,}018$	$-3{,}738 \pm 0{,}035$	0,098
0,70	$-2{,}926 \pm 0{,}043$	$-3{,}753 \pm 0{,}046$	0,096
1,00	$-3{,}064 \pm 0{,}049$	$-3{,}808 \pm 0{,}075$	0,087

Cuadro 6: Estudio de α en BLX — S&P 500 ($N = 457$)

α	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,00	$-2{,}996 \pm 0{,}080$	$-3{,}983 \pm 0{,}137$	0,012
0,10	$-2{,}983 \pm 0{,}067$	$-3{,}942 \pm 0{,}116$	0,013
0,20	$-2{,}944 \pm 0{,}057$	$-3{,}932 \pm 0{,}092$	0,018
0,30	$-2{,}877 \pm 0{,}043$	$-3{,}920 \pm 0{,}083$	0,014
0,50	$-2{,}682 \pm 0{,}029$	$-3{,}893 \pm 0{,}067$	0,019
0,70	$-2{,}737 \pm 0{,}034$	$-3{,}893 \pm 0{,}064$	0,018
1,00	$-2{,}790 \pm 0{,}041$	$-3{,}894 \pm 0{,}082$	0,019

7.2. Boxplots del estudio

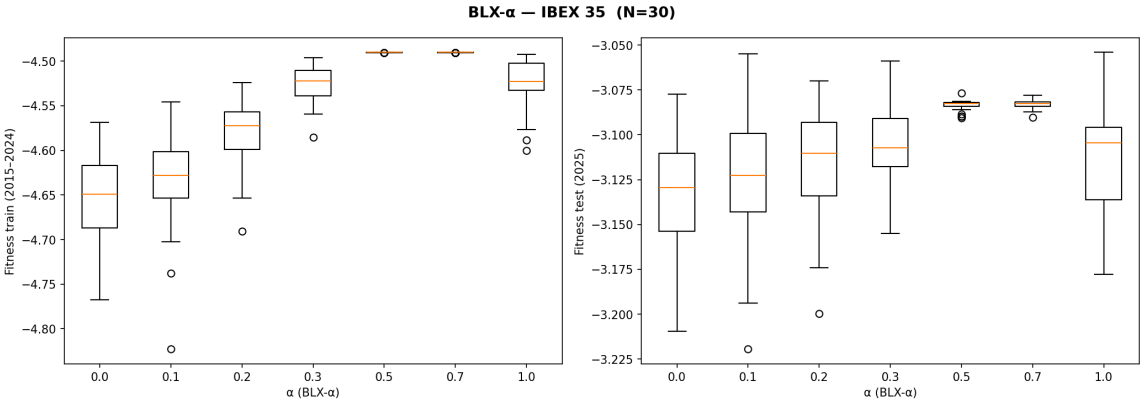


Figura 7: Distribución del fitness (train y test 2025) por valor de α — IBEX 35.

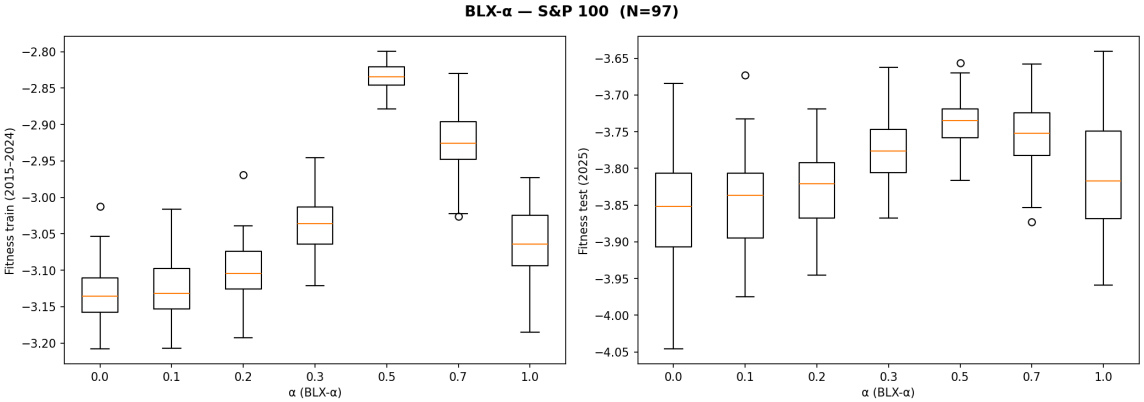


Figura 8: Distribución del fitness por α — S&P 100.

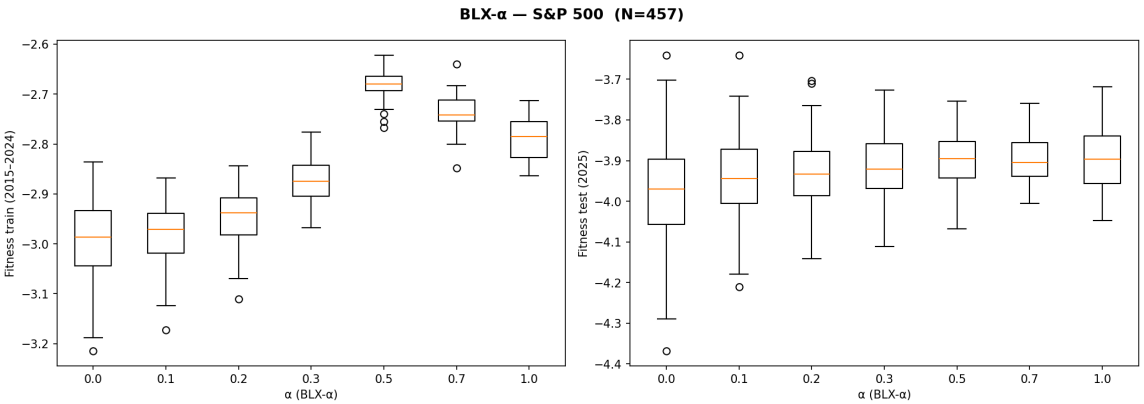


Figura 9: Distribución del fitness por α — S&P 500.

7.3. Análisis

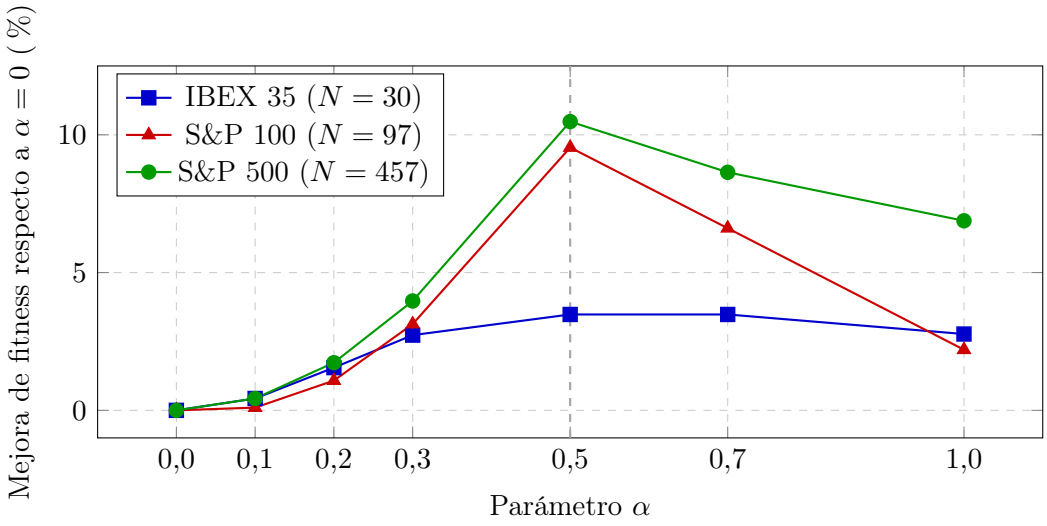


Figura 10: Mejora porcentual del fitness de entrenamiento respecto a $\alpha = 0$ en función de α, para los tres datasets. La línea discontinua marca el óptimo en $\alpha = 0{,}5$.

La Figura 10 muestra la mejora porcentual del fitness respecto a $\alpha = 0$ en función de α para los tres datasets. La forma de **U invertida** es visible en los tres casos, aunque con perfiles distintos:

- **IBEX 35 (azul,** $N = 30$**)**: curva muy plana. La mejora máxima es solo del 3,5 % y la curva hace plateau en $\alpha \in [0{,}5, 0{,}7]$ con apenas descenso en $\alpha = 1{,}0$. Con un espacio de soluciones pequeño ($N = 30$), incluso α bajos producen exploración suficiente; valores altos no degradan significativamente la calidad.
- **S&P 100 (rojo,** $N = 97$**)**: U invertida más pronunciada y clásica. El pico es del 9,5 % en $\alpha = 0{,}5$ y la caída a $\alpha = 1{,}0$ es muy marcada (solo 2,2 % de mejora restante). Es el dataset donde la elección de α tiene mayor impacto relativo.
- **S&P 500 (verde,** $N = 457$**)**: pico más alto (10,5 %) pero la bajada posterior es más suave (6,9 % aún en $\alpha = 1{,}0$). El gran espacio de soluciones aguanta más exploración antes de que sea perjudicial; la zona óptima es más ancha.

$\alpha = 0{,}5$ **es óptimo en los tres datasets**. El valor $\alpha = 0{,}3$ recomendado en el guion es subóptimo: $\alpha = 0{,}5$ mejora el fitness en $\approx 7\,\%$ en syp_100 ($-3{,}035$ vs $-2{,}834$) y syp_500 ($-2{,}877$ vs $-2{,}682$).

Un hallazgo contraintuitivo es que σ **disminuye al aumentar** α hasta el óptimo: más exploración produce resultados más consistentes entre semillas. Con α bajo la población converge a distintos óptimos locales según la semilla (alta varianza); con α adecuado escapa de esos óptimos y converge siempre hacia el mismo atractor. El caso extremo es ibex_35 con $\alpha \in \{0{,}5, 0{,}7\}$: $\sigma = 0{,}000$, prácticamente determinístico en las 50 semillas.

Finalmente, $\alpha = 0{,}0$ reproduce el comportamiento del cruce Aritmético (sin extensión del intervalo), confirmando que la ventaja de BLX proviene exclusivamente de la exploración más allá del convex hull de los padres.